

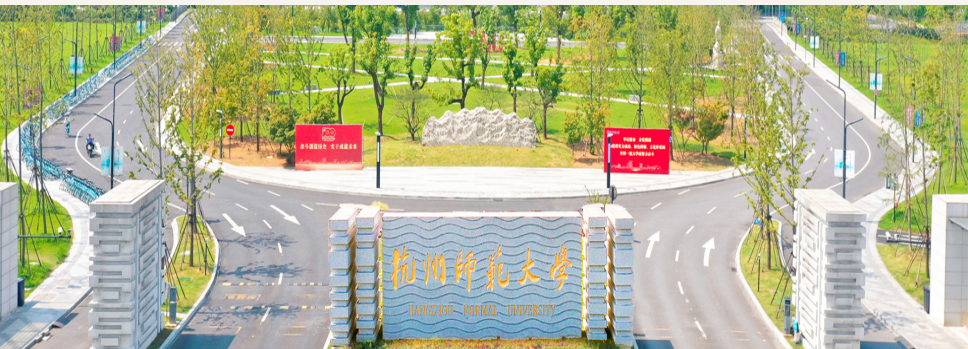
第二章 - 变量、数据类型、表达式 (编程练习)

张建章

阿里巴巴商学院

杭州师范大学

2026-09



1 变量与赋值

2 基本数据类型

3 表达式与类型转换

用变量记录一笔跨境订单

订单中的业务数据

- 订单编号: CN-2026-0908
- 商品名称: 蓝牙耳机
- 商品单价: 29.90美元
- 购买数量: 3件
- 优惠金额: 5.00美元
- 管理汇率: 7.12

在Python程序中记录数据

```
order_id = "CN-2026-0908"  
product_name = "蓝牙耳机"  
  
unit_price_usd = 29.90  
quantity = 3  
discount_usd = 5.00  
exchange_rate = 7.12
```

程序首先用**有意义的名称**记录完成任务所需的数据。

观察：每一行代码可以分成哪三个部分？

变量与赋值

变量是程序中用于引用数据的名称。初学时可以把变量理解为 **给一项数据起名字**，方便程序重复使用这项数据。

```
unit_price_usd = 29.90
quantity = 3

subtotal_usd = unit_price_usd * quantity
```

赋值语句的三个部分

- 左侧：变量名
- 中间：赋值符号 `=`
- 右侧：数据或计算表达式

Python的执行过程

- ① 读取右侧变量的当前值
- ② 计算右侧表达式
- ③ 将结果赋给左侧变量

赋值语句的阅读方法

`=` 表示赋值。Python先计算右侧，再把结果交给左侧变量。

变量的值会随程序运行而改变

某商品当前有50件库存，随后连续售出3件和5件：

```
stock = 50
stock = stock - 3
stock = stock - 5

stock
```

次序	执行的语句	stock 当前值
1	stock = 50	50
2	stock = stock - 3	47
3	stock = stock - 5	42

执行 `stock = stock - 3` 时，Python先读取 `stock` 原来的值50，计算 `50 - 3`，再将结果47重新赋给 `stock`。

Notebook中的变量状态

变量的当前值由代码的**实际运行顺序**决定。如果运行结果与预期不同，可以重新启动内核，再从上到下运行全部单元格（点那个快进按钮）。

变量命名

Python的语法要求

- 可以使用字母、数字和下划线
- 不能以数字开头
- 不能包含空格、连字符等符号
- 不能使用Python关键字
- 区分大小写，例如price和Price是两个变量

本课程的命名习惯

- 使用小写英文单词
- 多个单词之间使用下划线
- 名称应准确表达业务含义
- 不能使用Python保留关键字，例如，for、if、return等
- 避免覆盖Python内置名称，例如，str、float、sum。

`unit_price_usd``exchange_rate``amount_cny`

好的变量名使代码接近一份**可以阅读的业务计算说明**。

变量命名练习

下面左侧的变量名存在哪些问题？右侧是一种参考修改。

需要修改

```
2nd_order_id = "A002"
```

```
unit-price = 29.90
```

```
exchange rate = 7.12
```

```
float = 5.00
```

```
a = 84.70
```

一种参考修改

```
second_order_id = "A002"
```

```
unit_price_usd = 29.90
```

```
exchange_rate = 7.12
```

```
discount_usd = 5.00
```

```
amount_usd = 84.70
```

- 前三个变量名违反Python的命名语法，程序无法运行
- `float` 和 `a` 语法有效，但无法清楚表达业务含义

高质量变量名同时满足：语法有效，业务含义清楚。

一笔订单包含不同类型的数据

数据类型说明Python应当如何理解一项数据，以及可以对它进行哪些操作。

```
order_id = "CN-2026-0908"  
quantity = 3  
unit_price_usd = 29.90  
is_paid = True  
refund_reason = None
```

- `int`：整数
- `float`：浮点数
- `str`：字符串
- `bool`：布尔值
- `None`：表示没有值

选择数据类型的依据

一项数据的业务含义和后续操作共同决定它的类型。订单编号虽然包含数字，但它用于标识订单，因此应当保存为字符串。

严格地说，`None` 是Python中的一个特殊值，它的类型是 `NoneType`。

整数与浮点数

整数 `int`

```
quantity = 3
stock = 50
order_count = 128

type(quantity)  # int
```

常见业务数据包括：

- 商品数量
- 库存数量
- 订单数量

浮点数 `float`

```
unit_price_usd = 29.90
exchange_rate = 7.12
discount_rate = 0.15

type(unit_price_usd)  #
↪ float
```

常见业务数据包括：

- 商品价格和汇率
- 折扣率和增长率
- 平均值

浮点数的近似表示（浮点数的不定尾数现象）

计算机使用二进制近似保存浮点数，许多十进制小数无法被精确表示。例如， $0.1 + 0.2$ 的结果可能显示为 `0.30000000000000004`。

字符串

字符串 `str` 用于表示文本。字符串内容需要放在单引号或双引号中。

```
order_id = "CN-2026-0908"  
product_name = "蓝牙耳机"  
country_code = "CN"  
product_code = "000872"  
  
type(order_id)  # str
```

订单编号、商品编码等即使全部由数字组成，通常也应当保存为字符串，因为它们用于**标识**，不用于算术计算。

```
quantity = 3  
quantity_text = "3"
```

`3` 表示数量，`"3"` 表示文本字符。

引号属于Python语法

变量名不加引号，**字符串内容都需要加引号**。单引号、双引号、三引号都可以表示字符串，但两端必须配对。

布尔值与None

布尔值 bool

布尔值用于表示只有两种可能结果的状态，取值为 `True` 或 `False`。

```
is_paid = True
is_refunded = False
has_coupon = True

type(is_paid) # bool
```

变量名通常写成可以回答“是或否”的问题，例如“是否付款”。

特殊值 None

`None` 表示当前没有可用的值，可以表示尚未填写、尚未发生或不适用。

```
payment_time = None
refund_reason = None

type(refund_reason) #
↪ NoneType
```

`None` 与字符串 `"None"` 含义不同。后者只是由四个字母组成的文本。

书写规则

`True`、`False` 和 `None` 的**首字母必须大写**，并且不需要添加引号。

用type()检查数据类型

内置函数 `type()` 可以查看一项数据的类型。

```
product_code = "000872"  
quantity = 3  
discount_rate = 0.10  
is_member = False  
coupon_code = None
```

```
type(product_code) # str  
type(quantity) # int  
# float  
type(discount_rate)  
type(is_member) # bool  
# NoneType  
type(coupon_code)
```

选择类型的基本思路

需要算术计算的数据使用 `int` 或 `float`；文本和编码使用 `str`；两种状态使用 `bool`；暂时没有值时可以使用 `None`。

算数表达式

表达式由**数据**和**操作符**组成。Python计算表达式后会得到一个值。算数表达式进行算数运算。

表 1: 算数操作符

操作符	含义及示例
+	加法: 收入加运费
-	减法: 总价减优惠
*	乘法: 单价乘数量
/	除法: 销售额除以订单数
//	整除: 商品可以装满几箱
%	取余: 装箱后还剩几件
**	幂运算: 计算复合增长

```
unit_price_usd = 29.90
quantity = 3
discount_usd = 5.00
```

```
subtotal_usd =
↳ unit_price_usd *
↳ quantity
```

```
amount_usd = subtotal_usd
↳ - discount_usd
```

- `unit_price_usd * quantity` 是表达式, 表达式计算后得到商品总价
- 赋值语句 `subtotal_usd = unit_price_usd * quantity` 把结果交给变量

比较表达式与逻辑表达式

比较表达式

比较两个值，计算结果为True或False。

操作符	含义
<code>==</code>	等于
<code>!=</code>	不等于
<code>></code>	大于
<code><</code>	小于
<code>>=</code>	大于或等于
<code><=</code>	小于或等于

```
amount_usd = 84.70
quantity = 3
amount_usd >= 100  # False
quantity == 3      # True
```

逻辑表达式

逻辑操作符可以组合多个条件。

- `and`：两个条件都成立
- `or`：至少一个条件成立
- `not`：反转原来的结果

```
amount_usd = 120
quantity = 4
is_paid = True
is_refunded = False
is_large_order = (
    amount_usd >= 100
    and quantity >= 3
)
can_ship = is_paid and not
    ↪ is_refunded
```

`=` 把右侧结果赋给变量，`==` 比较左右两侧的值是否相等。

类型转换

类型转换使用内置函数把一个值转换为另一种类型。

```
quantity_text = "3"
quantity = int(quantity_text)

unit_price_text = "29.90"
unit_price_usd =
↪ float(unit_price_text)

order_number = 20260908
order_id = str(order_number)
```

- `int()` 转换为整数
- `float()` 转换为浮点数
- `str()` 转换为字符串

类型转换会**产生一个新值**。原来的变量仍然保留原来的值和类型。

```
type(quantity_text) #
↪ str
type(quantity) # int
```

字符串必须符合目标类型的格式

`int("3")` 和 `float("29.90")` 可以正常转换。

`int("three")` 和 `float("29.90美元")` 会产生错误。

从网页表单、文件或其他系统获得的数据经常是字符串。参与计算之前，需要先把文本转换为合适的数值类型。

```
# 原始数据都是字符串
unit_price_text = "29.90"
quantity_text = "3"
discount_text = "5.00"
# 转换为可以计算的数值
unit_price_usd =
    ↪ float(unit_price_text)
quantity = int(quantity_text)
discount_usd = float(discount_text)
# 按照业务规则计算
subtotal_usd = unit_price_usd *
    ↪ quantity
amount_usd = round(
    subtotal_usd - discount_usd, 2
)
amount_usd # 84.7
```

阅读代码时依次检查

- ❶ 原始数据是什么类型？
- ❷ 每项数据需要转换为什么类型？
- ❸ 表达式是否符合业务规则？
- ❹ 计算结果是否合理？

尝试回答

- 如果不进行类型转换，会发生什么？
- 数量可以使用 `float` 吗？
- 优惠金额大于商品总价时，结果合理吗？

上例总结：先确认数据含义和类型，再按照业务规则编写（算数）表达式。

未完待续